

AGRIAI 2.0 | FORENSIC SOFTWARE EXAMINATION

Code Path Atlas

File-by-file execution maps from a farmer click to provider, storage and admin control

Volume 04

Prepared and authored solely by Thaddeus Nii Teiko Tagoe

Founder, designer and sole creator of AgriAI

Computer Science, University of Ghana

Repository examined: ThaddeusHackz/agriai-v2

Evidence commit: 991457272b7fec548e02c645c5593b930af5ea29

Examination date: 18 August 2026

Classification: Internal technical record

Technical basis

This document is grounded in direct source review, route enumeration, dependency audit, secret pattern scanning, strict TypeScript checking, ESLint, production compilation, PostgreSQL integration tests and local HTTP behavior tests. Live third party AI accounts were not available in the examination environment, so provider success is assessed from code paths and contracts rather than a claim of live production availability.

1. Reading method

Every path below begins with an observable user action and follows the actual imported functions. File and line references use the examined commit. The diagrams distinguish client state, route orchestration, provider calls and persistence.

2. Path A: farmer sends a question

```
components/Chat.tsx:88 sendMessage
  -> POST /api/chat with message, language, mode, webSearch, history, sessionId
app/api/chat/route.ts:32 POST
  -> lib/db.ts trackQuestion and trackMessage
  -> optional lib/search.ts searchWeb and contextBlock
  -> lib/db.ts getDB settings.chat
  -> lib/languages.ts languageInstruction
  -> lib/ai.ts geminiGenerateStream
    -> @google/genai generateContentStream
    -> SSE delta for each chunk
  -> on failure lib/cloudflare.ts cloudflareChat
    -> Cloudflare Workers AI REST Llama
  -> on failure lib/ai.ts localAnswer
    -> db.knowledge then regex answers then generic card
  -> lib/db.ts mutate conversation
    -> atomic data/db.json
    -> lib/pg-store.ts schedulePostgresSave
  -> SSE done
components/Chat.tsx
  -> render Markdown, sources, provider and demo state
```

3. Path B: optional live search

```
webSearch=true
  -> searchWeb(message + " Ghana 2026")
    if TAVILY_API_KEY:
      POST api.tavily.com/search, advanced, six results
      on success return answer and sources
    on no key or error:
      GET Wikipedia OpenSearch
      GET DuckDuckGo Instant Answer
  -> contextBlock numbers snippets
  -> append block into the model system instruction
  -> return source title and URL in SSE done
```

4. Path C: crop photo diagnosis

```
components/DiseaseDetector.tsx
  -> FileReader data URL
  -> POST /api/vision
app/api/vision/route.ts
  -> type, length and language checks
  -> lib/ai.ts detectCropDisease
    -> parse data URL
    -> get settings.chat.visionModel
    -> Gemini multimodal prompt and inlineData
    -> extract first broad JSON object
    -> clamp confidence, cap treatment to six
    -> fallback demo card on missing key or all failures
  -> JSON response
```

5. Path D: agricultural image generation

```
components/CropStudio.tsx
  -> POST /api/generate {prompt}
app/api/generate/route.ts
  -> provider configured check
  -> 500-char cap
  -> SAFE_SUBJECTS substring check
  -> add photographic suffix
  -> lib/cloudflare.ts cloudflareImage
      -> Flux-1-schnell, four steps
  -> if failed lib/ai.ts geminiImage
      -> three image model names in order
  -> JSON image data URL
components/CropStudio.tsx
  -> img preview
  -> anchor download
```

6. Path E: voice input

```
components/Chat.tsx microphone
  if SpeechRecognition exists:
    browser converts speech to text
  else:
    getUserMedia -> MediaRecorder -> Blob
    POST /api/transcribe multipart form
    route buffers File -> base64
    lib/ai.ts geminiGenerateText, preferred gemini-2.5-flash
    response text is inserted into chat input
```

7. Path F: voice output

```
components/Chat.tsx speak(answer)
  -> POST /api/tts {text, language, speed}
  -> choose fixed ElevenLabs voice ID
  -> POST ElevenLabs text-to-speech
  -> audio/mpeg
  -> browser Blob URL and Audio.play()
```

8. Path G: weather

```
components/WeatherSection.tsx
  -> GET /api/weather?city=<selection>
app/api/weather/route.ts
  -> lib/openweather.ts openweatherForecast
      -> current and forecast endpoints in parallel
      -> aggregate 3-hour entries into days
  -> on failure Open-Meteo current and daily
  -> on failure deterministic seedForecast
  -> adviceFor(temp, precip, humidity)
  -> component cards
```

9. Path H: admin login

```
app/admin/login/page.tsx
  -> POST /api/admin/login
app/api/admin/login/route.ts
  -> lib/auth.ts clientId
  -> isRateLimited
  -> findUserByEmail from lib/db.ts
  -> bcrypt compareSync
  -> randomBytes(32) session token
  -> mutate Database.sessions
  -> Set-Cookie agriai_session
components/admin/AdminShell.tsx
  -> GET /api/admin/me
  -> authenticated tabs
```

10. Path I: admin changes model behavior

```
components/admin/AITab.tsx
-> edits model, visionModel, temperature, maxTokens and prompts
-> components/admin/api.ts
-> POST /api/admin/settings
-> getSessionUser accepts admin or editor
-> merge patch.chat into settings.chat
-> lib/db.ts mutate and persist
next chat request
-> reads settings.chat immediately
-> no rebuild or restart required
```

11. Path J: boot and persistence

```
Next.js server boot
-> instrumentation.ts register
-> if DATABASE_URL:
  initPostgres creates agriai_state
  hydrateFromPostgres only when data/db.json is absent
  -> SELECT doc WHERE id=1
  -> write local JSON and set in-process cache
first ordinary DB access
-> getDB -> load data/db.json
-> if absent or invalid -> defaultDB seed version 4
mutation
-> callback changes in-process object
-> write db.json.tmp then rename db.json
-> schedule full JSONB upsert after 1500ms
```

12. Source file roster

File	Role	Change impact
lib/env.ts	Provider aliases and status	All provider configuration
lib/ai.ts	Gemini, vision, image and local knowledge	Every AI response tier
lib/cloudflare.ts	Llama and Flux REST	Chat fallback and Studio primary
lib/search.ts	Grounding retrieval	Citations and current information
lib/languages.ts	Language labels and instructions	Multilingual steering
app/api/chat/route.ts	Chat orchestration	Main AI behavior and persistence
app/api/vision/route.ts	Vision HTTP validation	Disease upload boundary
app/api/generate/route.ts	Image subject gate	Studio safety and provider order
app/api/transcribe/route.ts	Audio ingestion	Server speech input
app/api/tts/route.ts	ElevenLabs output	Speech privacy and voice mapping
components/Chat.tsx	Farmer conversation UI	SSE, voice, feedback and control state
components/DiseaseDetector.tsx	Photo UI	Image preprocessing and diagnosis display
components/CropStudio.tsx	Image UI	Prompt and download workflow
lib/db.ts	State and seed	All settings, records, users and analytics
lib/pg-store.ts	Durability mirror	PostgreSQL consistency
lib/auth.ts	Identity	Admin access and session security
lib/types.ts	Compile-time schema	Cross-module shape changes
instrumentation.ts	Boot hydration	Restart recovery
next.config.ts	Headers and development origins	Browser hardening
render.yaml	Managed production topology	Secrets, database and deployment

13. Change impact map

Desired change	Primary files	Mandatory regression checks
Add an AI provider	lib/provider adapter, chat route, env.ts	Fallback order, abort, attribution, empty output
Add a language	languages.ts, chat and vision whitelist, TTS mapping	Prompt steering, UI label, audio, local fallback
Change persistence	db.ts, pg-store.ts, instrumentation.ts	Atomicity, concurrency, hydration, sessions
Change admin role	auth.ts and every admin route	Server authorization, hidden tabs, destructive operations
Change chat SSE	chat route and Chat.tsx	Frame parsing, partial text, abort and final replacement
Change disease schema	ai.ts, vision route, DiseaseDetector.tsx	JSON parser, confidence and treatments

Authorship and control statement

This volume was prepared, authored and issued solely under the name of Thaddeus Nii Teiko Tagoe. This atlas makes the complete execution path traceable from user action to the exact implementation unit. The repository records identify Thaddeus Tagoe as the founder, sole designer, developer and maintainer of AgriAI.